



Database Management Systems

UNIT 2

By Santoshh s potti



Contents

- **Chapter 1- Relational Database Model:** Logical view of data, keys, integrity rules, Relational Database design: features of good relational database design, atomic domain and Normalization (1NF, 2NF, 3NF, BCNF).
- **Chapter 2-Relational Algebra and Calculus:** Introduction, Selection and projection, set operations, renaming, Joins, Division, syntax, semantics. Operators, grouping and ungrouping, relational comparison. Calculus: Tuple relational calculus, Domain relational Calculus, calculus vs algebra, computational capabilities



Introduction

- Tables / Relations are logical structure which is a collection of 2-dimensional tables consisting of horizontal rows and vertical columns.
- It is an abstract concept and do not represent how data is stored in physical memory of computer system.
- Each table in database has its own unique table name by which its contents can be referred.

Characteristics



A table is perceived as 2-dimensional structure composed of rows and columns.



Each table row(tuple) represents a single entity occurrence within the entity set.



Each table column represents an attribute and each column has a distinct name.



Each row/column intersection represents a single data value.

Characteristics contd..



All values in a column must confirm to the same data format.



Each column has specific range of values known as attribute domain.



The order of the rows and columns is immaterial to DBMS.



Each table must have an attribute or a combination of attributes that uniquely identifies each row.

Attributes

Each column in the table represents one data item stored in database for that table.

Such column in database is called as attribute of a table.

Tables must have at least one column in it and no two columns can have same name.

The ANSI/ISO SQL standard does not specify a maximum number of columns in a table.

Tuple/Records

A single row or tuple contains all the information about a single entity.

Each horizontal row of the table represents a single entity.

A table can have any number of rows from zero to thousand.

If number of rows are zero then it is called as empty table.

1.2 Key

The column value that uniquely identifies a single record in the table is called as KEY of table.

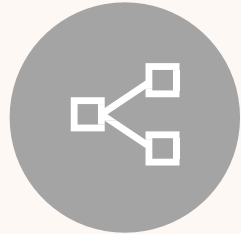
An attribute or set of attributes whose values uniquely identify each entity in an entity set is called as key for that entity set.

Any key consisting of single attribute is called a simple key while that consisting of a combination of attributes is called a composite key.

Types of Keys



SUPER KEY



CANDIDATE
KEY



PRIMARY KEY



FOREIGN KEY



SECONDARY
KEY

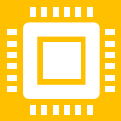
Example:



Consider a relation



Student(Gr_no, Roll_no, Fname, Lname, DOB, Phone, Address), and



Result(Gr_no, sub_code, Marks)

1. Super Key



- A super key is any combination of fields within a table that uniquely identifies each record within that table.



- In other words, A key attribute with additional attributes that uniquely identifies a single record in a table.



- In above relation Student, Gr_no and Roll_no forms a super key as each student has a unique Gr_no.

2.Candidate Key



A candidate key is a subset of super key



A candidate key is a single field or the least combination of fields that uniquely identifies each record in the table



The least combination of fields distinguishes a candidate key from a super key



In relation student Gr_no and Roll_no are the candidate keys as they uniquely identifies the records.

3. Primary Key



A primary key is a candidate key that is most appropriate to be the main reference key for the table.



As its name suggests, it is the primary key of reference for the table and is used throughout the database to help establish relationships with other tables.



As with any candidate key the primary key must contain unique values, must never be null and uniquely identify each record in the table.



Primary key= Unique+ Not Null



Primary keys are mandatory for every table each record must have a value for its primary key.



In student relation, Gr_no will be the most appropriate choice of primary key.

4. Foreign Key



A foreign key is generally a primary key from one table that appears as a field in another where the first table has a relationship to the second.



In other words, if we had a table A with primary key X that linked to a table B where X was a field in B then X would be a foreign key in B.



In relations Student and Result Gr_no is a primary key in student relation and foreign key in Result relation.

5. Secondary key

A table may have one or more choices for the primary key.

Collectively these are known as Candidate keys

One is selected as primary key. Those not selected are known as Secondary keys or alternative keys.

In relation student, if Gr_no is a primary key then Roll_no is a alternate key.



Relational Integrity Rules

In order to ensure that a database provides meaningful information, Integrity rules or constraints are necessary to implement.

Two important integrity rules called entity integrity and referential integrity are critical to maintain in any database.

Entity Integrity



Requirement: All entities are unique and no null entries in the primary key.



Purpose: Guarantees that each entity will have a unique identity.



Entity integrity means that each table must have a primary key attribute with the following characteristics:

No two rows of the table have the same value in the primary key column.
And A NULL value (i.e No data entry at all) is not permitted.

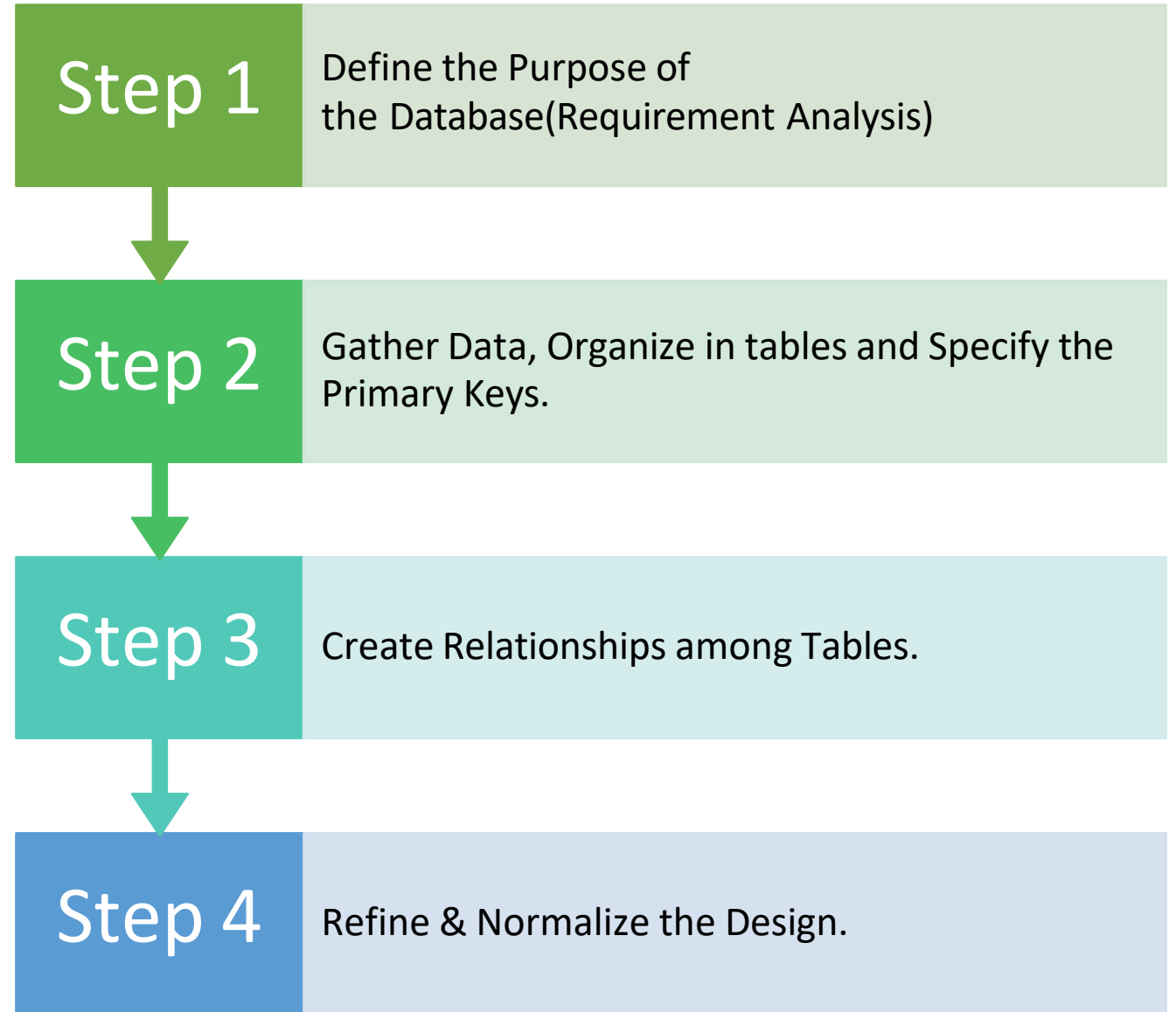
Referential Integrity

It can have a null entry as long as it is not a part of its tables primary key or an entry that matches the primary key value in a table to which it is related.

It is possible for an attribute NOT to have a corresponding value but it will be impossible to have an invalid entry.

The enforcement of referential integrity rule makes it impossible to delete a row in one table whose primary key has mandatory matching foreign key values in another table

Relational Database Design Process



Features Of Good Relational Database Design



The data should be centralised, and the information should be distributed to all the tables to reduce redundancy of data.



It should provide easy access to the data available in multiple tables.



It should take care of accuracy and integrity of information.



It should provide functionalities for data processing and reporting needs.



Data entry, updates and deletions should be efficient.



Data retrieval, summarization and reporting should also be efficient.



The database design must be self-documenting since much of the information is stored in the database rather than in the application.

Normalization

Step by step decomposition of complex relational tables into simple table.

Results in tables that represented in a simple manner and satisfy some constraints.

Avoids data redundancy by applying some constraints on data to avoid various data anomalies.

A normalized table is less vulnerable to data problems.

Process of designing a consistent database by minimizing redundancy and ensuring data integrity through the principle of non-loss decomposition.



Goals Of Normalization

- Ensures Data Integrity
 - Data integrity ensures the correctness of data stored within the database and can be achieved by imposing data integrity rules. An integrity rules restricts values present in the database
 - Prevents Redundancy
 - A non-normalized data is stored in different locations and hence modification makes data inconsistent. A normalized data stores data only in one place. Direct redundancy can result due to presence of same data in two different locations . Indirect redundancy results due to storing information that can be computed from the other data items stored within the database
 - Data Anomalies
 - Update Anomaly – Same information can be present in multiple records of various tables hence update to only one table will result in inconsistency
 - Insertion Anomaly – There is a possibility in which certain facts cannot be recorded at all or that are not yet recorded
 - Deletion Anomaly – Deletion of some data from a relation necessitates the deletion of unrelated data also
-

Disadvantages Of Normalization

Increases number of relations: As normalization involves the decomposition of relations into multiple relations or tables hence higher degrees of normalization typically involve more tables. Therefore, if highly normalized tables are used in database applications, then the application becomes complex.

Reduces performances: Higher degrees of normalization involve more tables and create the need for a larger number of joins which can reduce performance.

Some redundancies are unavoidable. While normalizing the tables data integrity should not be compromised.

Normal Forms



Forms are designed to logically address potential problems such as inconsistencies and redundancy in information stored in the database.



A database is said to be in one of the Normal Forms if it satisfies the rules required by that form as well as the previous form and it will also not suffer from any of the problems addressed by the form



Types of Normal Forms

First Normal Form (1NF)

Second Normal Form (2NF)

Third Normal Form (3NF)

Boyce Codd Normal Form (BCNF)

Fourth Normal Form (4NF)

Fifth Normal Form (5NF)

Un Normalized Data

Faculty Code	Faculty Name	Date Of Birth	Subject	Hours
100	Yogesh	17/07/64	DSA	16
			SS	8
			IS	12
101	Amit	24/12/72	MIS	16
			PM	8
			IS	12
102	Omprakash	03/02/80	PWRC	8
			PCOM	8
			IP	16
103	Nitin	28/11/66	DT	10
			PCOM	8
			SS	8
104	Mahesh	01/01/86	DT	10
			ADBMS	8
			PWRC	8

First Normal Form (1NF)



A relation is in 1NF if it contains exactly one value for each attribute



1NF states that attributes included in relation must have atomic values and that any attribute in tuple must have a single value from the domain of that attribute



The above table does not have any atomic values in the Subject column , hence it is in un normalized form



The above table in 1NF will be as follows

1NF Table

Sr No	Faculty Code	Faculty Name	Date Of Birth	Subject	Hours
1	100	Yogesh	17/07/64	DSA	16
2	100	Yogesh	17/07/64	SS	8
3	100	Yogesh	17/07/64	IS	12
4	101	Amit	24/12/72	MIS	16
5	101	Amit	24/12/72	PM	8
6	101	Amit	24/12/72	IS	12
7	102	Omprakash	03/02/80	PWRC	8
8	102	Omprakash	03/02/80	PCOM	8
9	102	Omprakash	03/02/80	IP	16
10	103	Nitin	28/11/66	DT	10
11	103	Nitin	28/11/66	PCOM	8
12	103	Nitin	28/11/66	SS	8
13	104	Mahesh	01/01/86	DT	10
14	104	Mahesh	01/01/86	ADBMS	8
15	104	Mahesh	01/01/86	PWRC	8

Second Normal Form:

- A relation is in 2NF if it is in 1NF and every non-key attribute is fully functionally dependent on primary key of the relation and not just part of the primary key
- 2NF prohibits partial dependencies
- Steps
 - Find and remove attributes that are related to only a part of the key
 - Group the removed attributes in another table
 - Assign the new table the key that consists of that part of the old composite key
- Hence in our example we will have two tables
 - Faculty (Faculty Code, Faculty Name, Date Of Birth)
 - Subject (Faculty Code, Subject, Hours)
- Anomalies
 - Inserting records of various faculties teaching the same subject results in redundancy of hours information
 - As number of hours is repeated any change done has to be repeated for every instance
 - If a faculty leaves the organization information regarding the subject is also lost

2NF

Table:

Non repeating groups

Faculty Code	Faculty Name	DOB
100	Yogesh	17/7/85
101	Amit	24/12/86
102	Om	02-03-85
103	Nitin	28/11/86
104	Mahesh	01-01-86

repeating groups

Sr No.	Faculty Code	Subject	Hours
1	100	DSA	16
2	100	SS	8
3	100	IS	12
4	101	MIS	16
5	101	PM	8
6	101	IS	12
7	102	PW	8
8	102	COM	8
9	102	IP	16
10	103	DT	10
11	103	PCOM	8
13	103	SS	8
14	104	DT	10
15	104	DBMS	8
16	104	IP	8

Third Normal Form

- This normal form is used to reduce transitive dependency
- If A is dependent on B and B is dependent on C then A is said to transitively dependent on C
- A relation is in 3NF if it is in 2NF and no non-key attribute of the relation is transitively dependent on the primary key
- 3NF prohibits transitive dependencies
- In the above table hours depends on Subject and Subject depends on Faculty Code. Hence there exists a transitive dependency between these three.
- If the faculty code is deleted the information regarding the subject is lost
- For a table to be in 3NF transitive dependency must be removed. So the 3NF table becomes
 - Faculty_Subject (Faculty Code, Subject)
 - Subject_Hours (Subject, Hours)
- Advantages
 - No redundancy of data for subject and hours while inserting records
 - Subject and hours are stored in separate table so Updation becomes easier
 - Even if faculty record is deleted subject hours can still be retrieved

• 3NF Form:

Fac_Sub

Sr No.	Faculty Code	Subject
1	100	DSA
2	100	SS
3	100	IS
4	101	MIS
5	101	PM
6	101	IS
7	102	PW
8	102	COM
9	102	IP
10	103	DT
11	103	PCOM
13	103	SS
14	104	DT
15	104	DBMS
16	104	IP

Sub_Hrs

Subject	Hours
DSA	16
SS	8
IS	12
MIS	16
PM	8
IS	12
PW	8
COM	8
IP	16
DT	10
PCOM	8
SS	8
DT	10
DBMS	8
IP	8

BCNF Normal Form

- The intention of BCNF is that 3NF does not satisfactorily handle the case of a relation processing two or more composite or overlapping candidate keys
- Candidate key is a column which has the ability to become a primary key
- A determinant is any attribute on which some other attribute is fully functionally dependent
- A relation is in BCNF if and only if every determinant is a candidate key
- Consider the following table in which there is a dependency between (SoldierId + OfficerId) and UnitId.
- There is also a dependency between UnitId and OfficerId which is not partial neither transitive
- What we have is a table where a determinate is not a candidate key UnitId. This problem can be solved by BCNF

BCNF Normal Form Contd..

SoldierId	OfficerId	UnitId
1	A	1
2	A	1
3	B	2

SoldierId	UnitId
1	1
2	1
3	2

OfficerId	UnitId
A	1
A	1
B	2

RELATIONAL ALGEBRA:

Relational algebra became popular after publication of E.F Codd's relational model data in 1970.

It is a procedural language useful for representing query execution plan,& relatively close to SQL.

Relational algebra is set of operations which accepts one relation & produce new relation as a result.

This query is applied to tables/relations & output is also a table/relation.

FUNDAMENTAL OPERATIONS

UNARY OPERATION

- ❖ Select (σ)
- ❖ Project (Π)
- ❖ Rename (ρ)

BINARY OPERATIONS

- ❖ Union ($\cup$)
- ❖ Set difference ($-$)
- ❖ Cartesian Product ($\times$)

Additional Operations

- ❖ Set Intersection operation ($\cap$)
- ❖ Join Operations ($\bowtie$)
- ❖ Division

Select Operation (σ)

A) OVERVIEW

- This operator is used to select some rows from table which satisfy particular selection condition given in selection operation.
- Selection operator selects a set of tuples that satisfy a selection condition.
- The selection operation is a unary operation which means it operates on one relation.
- It is denoted by Greek letter Sigma.

A) SYNTAX

σ <attribute_name><comparison_operator><constant_value>(<input_table_name>)

Where,

Attribute_name : Name of column in table

Comparison_operator : = , < , <= , > , >= , <>

C) EXAMPLE

Eid	Ename	Age
1	Suhas	15
2	Ajay	23
3	Rani	43
4	Raju	25
5	Sonal	55

σ age < 30 (Employee)

Eid	Ename	Age
1	Suhas	15
2	Ajay	23
4	Raju	25

Projection operation (Π)

A) OVERVIEW

- This operator is used for selecting some of many columns in table to display in result set.
- Projection operator can select column or set of columns of table to be display in output of query.
- We can select only few columns or all columns of a table as per requirements.
- This is unary relational operator having only one input table.
- It is denoted by Greek letter pi.

B) SYNTAX

Π <column_list> (<input_table_name>)

C) EXAMPLE

1. Find employee's id, salary and age from employee table.

$\Pi_{\text{Eid, Salary, Age}}(\text{Employee})$

Eid	Salary	Age
1	10000	35
2	12000	24
3	24000	25

2. Find names of all the employees having salary less than 20000.

$\Pi_{\text{ename}}(\sigma_{\text{sal} < 20000}(\text{Employee}))$

Rename Operation (ρ)

A) OVERVIEW

- Rename operator returns an existing relation under a new name.
- Rename is denoted by Greek letter rho.

B) SYNTAX

$\rho_{\langle \text{New_Name} \rangle}(\langle \text{input_table_name} \rangle)$

C) EXAMPLE

1. Rename emp to employee

Syntax: ρ employee(emp)

2. Find the salary of an employee whose name is Allen from the relation emp_details.

Syntax:

$\prod$ sal(σ ename="Allen"(ρ emp_details (emp)))

SET Operator

Introduction

SQL SET operator allow combining results from two or more SELECT statements or combines result set of multiple queries.

The result of two queries can be combined using the set operations union, intersection and set difference.

Requirements

SELECT statement of both queries must retrieve the same number of columns.

Data types of corresponding columns in each query must be of same type.

1. Union operation ($\cup$)

- UNION is symbolized by $\cup$ symbol. It includes all tuples that are in tables A or in B. It also eliminates duplicate tuples. So, set A UNION set B would be expressed as:
- Syntax $\leftarrow A \cup B$
- (Query Expression 1) $\cup$ (Query Expression 2)
- For a union operation to be valid, the following conditions must hold -
- Table A and Table B must have the same number of attributes.
- Attribute domains need to be compatible.
- Duplicate tuples should be automatically removed

Consider the following tables.

Table A		Table B	
column 1	column 2	column 1	column 2
1	1	1	1
1	2	1	3

$A \cup B$ gives

Table $A \cup B$	
column 1	column 2
1	1
1	2
1	3

Consider the following relations

Borrower(customer_name, loan_number)

Depositor(customer_name, account_number)

Customer(customer_name, streetno, city)

Example:

Qn. List all the customers who have either an account or a loan or both.

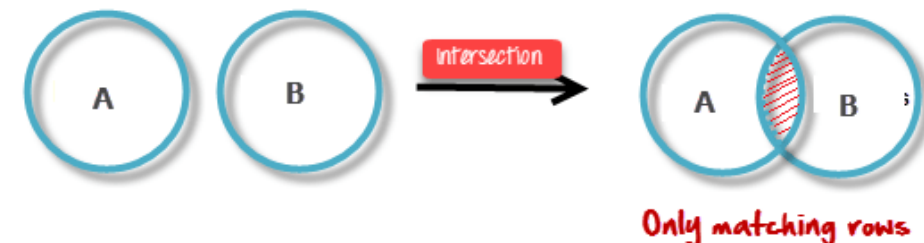
Syntax:

**Π customer_name(Borrower) $\cup$ Π
customer_name(Depositor)**

2. Intersection Operation

- An intersection is defined by the symbol $\cap$
- $A \cap B$
- Defines a relation consisting of a set of all tuple that are in both A and B.
- This operator finds out all rows that are common in both result of query 1 & in the result of query 2.
- Syntax: (Query Expression 1) $\cap$ (Query Expression 2)

Table $A \cap B$	
column 1	column 2
1	1





Example

Consider the following relations

Borrower(customer_name, loan_number)

Depositor(customer_name, account_number)

Customer(customer_name, streetno, city)

Qn. List all the customers who have both a loan and an account.

Syntax:

Π customer_name(Borrower) $\cap$ Π customer_name(Depositor)



3. Set Difference (-)

Table A - B	
column 1	column 2
1	2

- Returns all rows that are in the result of query 1 but not in the result of query 2
- (Query Expression 1)- (Query Expression 2)
- - Symbol denotes it. The result of $A - B$, is a relation which includes all tuples that are in A but not in B.
- The attribute name of A must match with the attribute name in B.
- It should be defined relation consisting of the tuples that are in relation A, but not in B.

Example

Consider the following relations

Borrower(customer_name, loan_number)

Depositor(customer_name,
account_number)

Customer(customer_name, streetno, city)

Qn. Find the names of all customers who have an account but not a loan.

Syntax:

**Π customer_name(Depositor) - Π
customer_name(Borrower)**

Cartesian product(X) or Cross Join

A cross join performs relational product or Cartesian product of two tables specified in query.

In this case every row in first table will be joined with every row in second. So finally, number of rows in result table will be equals to product of number of rows in table 1 and number of rows in table2

Syntax: (Query Expression 1)×(Query Expression 2)

Cartesian Product Operation

Cartesian-Product Operation – Example

■ Relations r, s :

A	B
α	1
β	2

r

C	D	E
α	10	a
β	10	a
β	20	b
γ	10	b

s

■ $r \times s$:

A	B	C	D	E
α	1	α	10	a
α	1	β	10	a
α	1	β	20	b
α	1	γ	10	b
β	2	α	10	a
β	2	β	10	a
β	2	β	20	b
β	2	γ	10	b

Join Operations

- Join operator helps us to retrieve data from multiple tables or relations.
- Join operation denoted by $\bowtie$.
- Most common type of join is “Natural join” in which columns having same name in two table will be taken for joining tables.
- Syntax:
(<table_name>) <join condition> (table_name)
- There are various types of joins possible in relational algebra.

Types of Joins

1. NATURAL JOIN ($\bowtie$)

- Natural join can only be performed if there is a common attribute (column) between the relations. The name and type of the attribute must be same.

Example

Consider the following two tables

C	
Num	Square
2	4
3	9

D	
Num	Cube
2	8
3	27

C $\bowtie$ D	
---------------	--

C $\bowtie$ D		
Num	Square	Cube
2	4	8
3	9	27

2. Inner Join

In many cases, tables are joined according to search conditions that find only the rows with matching values ; this type of join is known as inner join.

Inner join joins two tables when there is at least one match between two tables.

INNER JOIN: example

Students		
stud_id	fname	group_id
1	Boris	2
2	Beksultan	2
3	Aynur	

Groups	
group_id	group_name
1	CSSE-122
2	CSSE-124

Result table for INNER JOIN		
stud_id	fname	group_name
1	Boris	CSSE-124
2	Beksultan	CSSE-124

3. OUTER JOIN



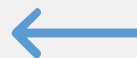
In an outer join the resultant table contains only the combinations of rows that satisfy the join conditions.



Rows that do not satisfy the join condition are discarded. Outer join joins two table although there is no match between two joining tables.



Outer joins are useful when you are trying to determine which values in related tables cause referential integrity problem.



Types

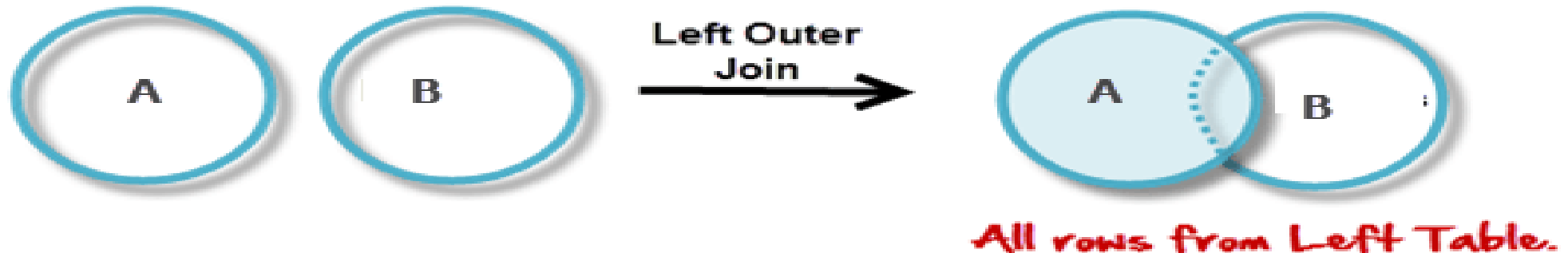
Left outer join

Right outer join

Full outer join

Left Outer Join($A \bowtie B$)

- Table on left side may contain null values. Left outer join takes all tuples in left relation that did not match with any tuple in the right relation.
- In the left outer join, operation allows keeping all tuple in the left relation. However, if there is no matching tuple is found in right relation, then the attributes of right relation in the join result are filled with null values.



Example

Consider the following 2 Tables

A	
Num	Square
2	4
3	9
4	16

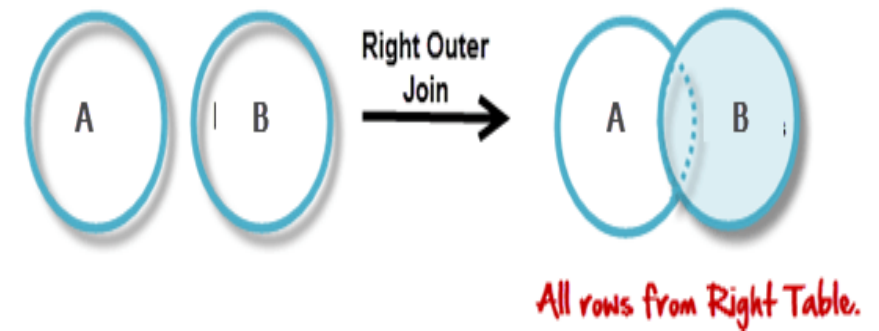
B	
Num	Cube
2	8
3	18
5	75

A  B

A ⋈ B		
Num	Square	Cube
2	4	8
3	9	18
4	16	-

Right Outer Join($A \bowtie B$)

- Table on right side may contain null values. Right outer join takes all tuples in right relation that did not match with any tuple in the left relation.
- In the right outer join, operation allows keeping all tuple in the right relation. However, if there is no matching tuple is found in the left relation, then the attributes of the left relation in the join result are filled with null values.



$A \bowtie B$

$A \bowtie B$		
Num	Cube	Square
2	8	4
3	18	9
5	75	-

Full Outer Join(A ⋈ B)

- Table on both sides of operator may contain null values. Full outer join takes all tuples in left relation that did not match with any tuple in the right relation and also involves all tuples in the right relation that did not match with any tuples in left relation.
- In a full outer join, all tuples from both relations are included in the result, irrespective of the matching condition.

A ⋈ B		
Num	Cube	Square
2	4	8
3	9	18
4	16	-
5	-	75

Relational Division Operator

A) OVERVIEW

- Produce the tuples in one relation r , that match all tuples in another relation s .
 - $R = (A_1, \dots, A_m, B_1, \dots, B_n)$
 - $S = (B_1, \dots, B_n)$
- It is denoted by $r \div s$
- $r \div s$ with attributes $A_1, \dots, A_m$ is the set of all tuples $\langle a \rangle$ such that for every tuple $\langle b \rangle$ in s , $\langle a, b \rangle$ is in r .

B) SYNTAX

(Query Expression 1) $\div$ (Query Expression 2)

Division Operation – Example

□ Relations r, s :

A	B
α	1
α	2
α	3
β	1
γ	1
δ	1
δ	3
δ	4
ϵ	6
ϵ	1
β	2

r

B
1
2

s

□ $r \div s$:

A
α
β

e.g.

A is customer name

B is branch-name

1 and 2 here show two specific branch-names

(Find customers who have an account in all branches of the bank)

Extended Relational Algebra operations

- **Generalized projection:**
- It extends the projection operation by allowing the arithmetic functions to be used in the projection list.
- Syntax: $\Pi F_1, F_2, \dots, F_n(E)$

Where E: Relational algebra expression

F_i : Arithmetic expression

For example: $\Pi \text{ customername, (Limit-credit_balance) (creditinfo)}$

Aggregate Functions

- It takes a collection of values and returns a single result.
- G sum(marks) (Record): Returns the sum of the marks attribute record.
- G max(marks) (Record): Returns maximum of the marks attribute record.
- G min(marks) (Record): Returns the minimum of the marks attribute record.
- G average(marks) (Record): Returns the average of the marks attribute record.
- G count(marks)(Record): Returns the count of the marks attribute.

ID	NAME	MARKS	AGE
1	Harsh	90	19
2	Suresh	50	20
3	Pratik	80	19
4	Dhanraj	95	21
5	Ram	85	18

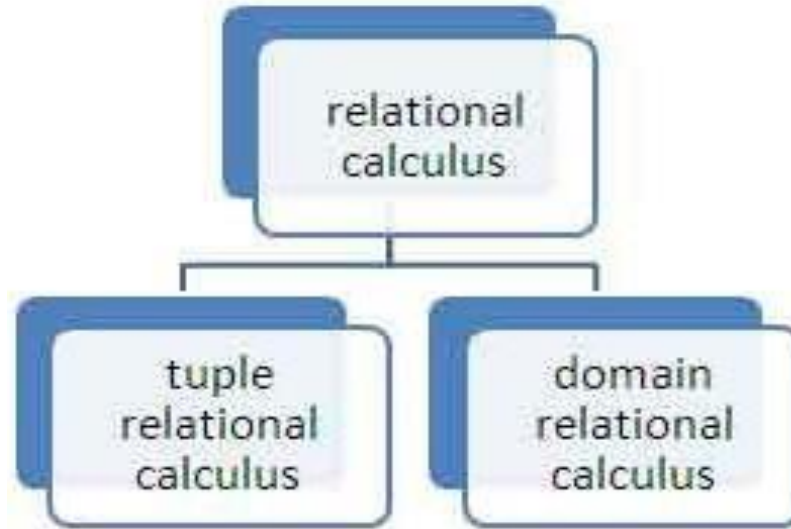
▪ Select	$\sigma_{\langle \textit{selection condition} \rangle}(R)$
▪ Project	$\pi_{\langle \textit{attribute list} \rangle}(R)$
▪ Rename	$\rho_{\langle \textit{new schema} \rangle}(R)$
▪ Union	$R \cup S$
▪ Intersection	$R \cap S$
▪ Difference	$R - S$
▪ Cross product	$R \times S$
▪ Join	$R \bowtie_{\langle \textit{join condition} \rangle} S$
▪ Natural join	$R \bowtie S$

Different Relational Algebra Operations

Relational Calculus

- Non procedural, declarative query language, that is, it tells what to do but never explains how to do it.
- Can define the information to be retrieved not any specific series of operations. In contrast, relational algebra defines the sequences of operations.
- Closer than relational algebra to how users would formulate queries in terms of their information needs, rather than in terms of operations.

- Involves variables that range over tuples or domain attributes.



- **Tuple Relational Calculus: Variables range over tuples.**
- **Domain Relational Calculus: Variables range over domain attributes.**
- Gives a way of referring to attributes of relations (and the specific tuples in them)

Tuple Relational Calculus

- A nonprocedural query language, where each query is of the form

$$\{t \mid P(t)\}$$

- It is the set of all tuples t such that predicate P is true for t .
- t is a *tuple variable*, $t[A]$ denotes the value of tuple t on attribute A .
- $t \in r$ denotes that tuple t is in relation r .
- P is a *formula* similar to that of the predicate calculus.

Predicate Calculus Formula

1. Set of attributes and constants
2. Set of comparison operators: (e.g., $<$, $\leq$, $=$, $\neq$, $>$, $\geq$)
3. Set of connectives: and ($\wedge$), or ($\vee$), not ($\neg$)
4. Implication ($\Rightarrow$): $x \Rightarrow y$, if x is true, then y is true
$$x \Rightarrow y \equiv \neg x \vee y$$
5. Set of quantifiers:
 - $\exists t \in r (Q(t)) \equiv$ "there exists" a tuple t in relation r such that predicate $Q(t)$ is true
 - $\forall t \in r (Q(t)) \equiv Q$ is true "for all" tuples t in relation r

Banking Example

branch (branch_name, branch_city, assets)

customer (customer_name, customer_street, customer_city)

account (account_number, branch_name, balance)

loan (loan_number, branch_name, amount)

depositor (customer_name, account_number)

borrower (customer_name, loan_number)

Example Queries

Qn. 1) Find the *loan_number*, *branch_name*, and *amount* for loans of over \$1200

- $\{t \mid t \in \text{loan} \wedge t[\text{amount}] > 1200\}$

Qn. 2) Find the loan number for each loan of an amount greater than \$1200

- $\{t \mid \exists s \in \text{loan} (t[\text{loan_number}] = s[\text{loan_number}] \wedge s[\text{amount}] > 1200)\}$

Qn. 3) Find the names of all customers having a loan, an account, or both at the bank

- $\{t \mid \exists s \in \text{borrower} (t[\text{customer_name}] = s[\text{customer_name}]) \vee \exists u \in \text{depositor} (t[\text{customer_name}] = u[\text{customer_name}])\}$

Q. 4) Find the names of all customers who have a loan and an account at the bank

- $\{t \mid \exists s \in \text{borrower} (t[\text{customer_name}] = s[\text{customer_name}]) \wedge \exists u \in \text{depositor} (t[\text{customer_name}] = u[\text{customer_name}])\}$

Q. 5) Find the names of all customers having a loan at the Perryridge branch.

- $\{t \mid \exists s \in \text{borrower} (t[\text{customer_name}] = s[\text{customer_name}] \wedge \exists u \in \text{loan} (u[\text{branch_name}] = \text{"Perryridge"} \wedge u[\text{loan_number}] = s[\text{loan_number}]))\}$

Domain Relational Calculus

- A nonprocedural query language equivalent in power to the tuple relational calculus.
- Each query is an expression of the form:

$$\{ \langle x_1, x_2, \dots, x_n \rangle \mid P(x_1, x_2, \dots, x_n) \}$$

- $x_1, x_2, \dots, x_n$ represent domain variables
- P represents a formula similar to that of the predicate calculus

Example Queries

Q1) Find the *loan_number*, *branch_name*, and *amount* for loans of over \$1200

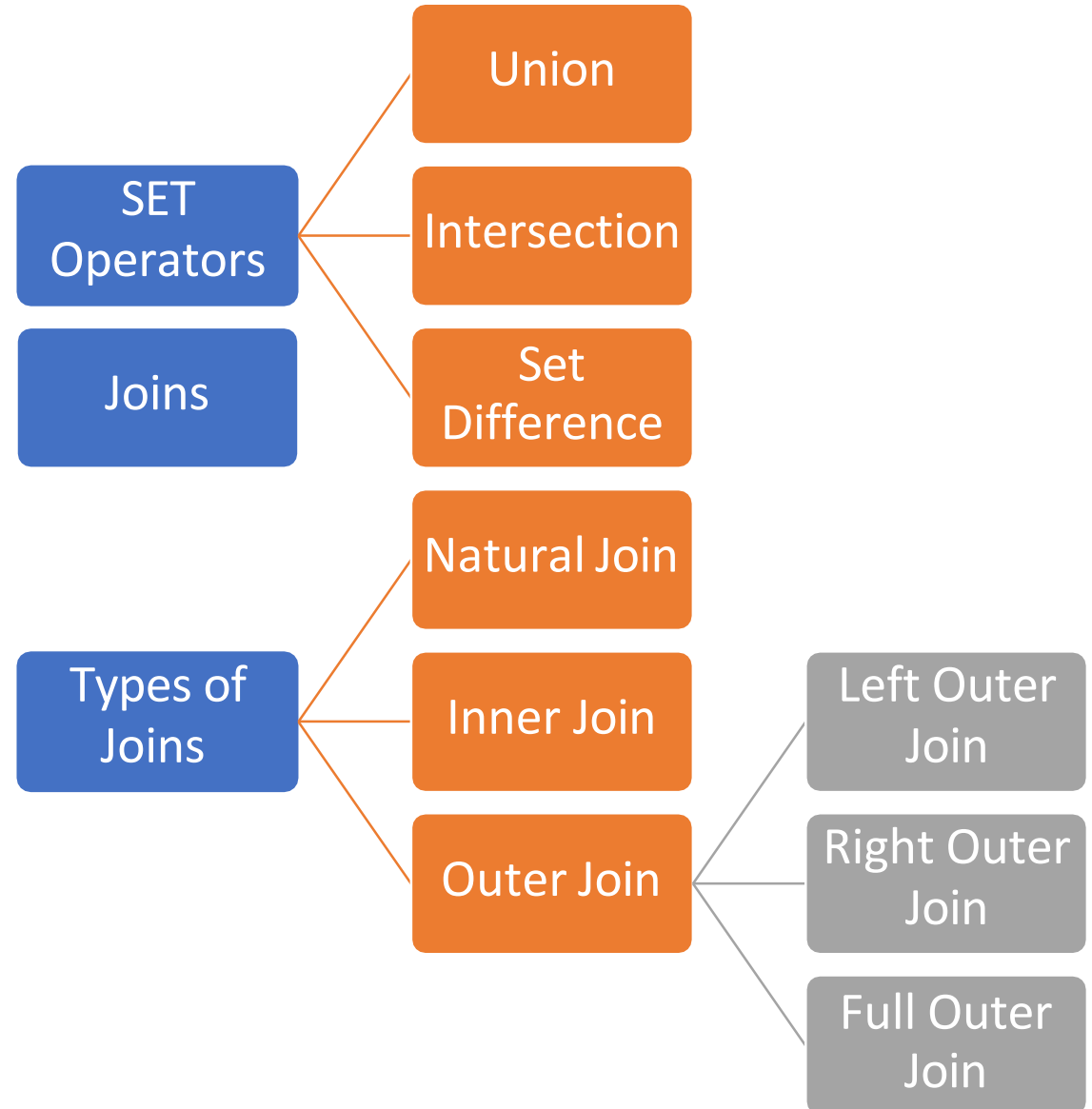
- $\{ \langle l, b, a \rangle \mid \langle l, b, a \rangle \in \text{loan} \wedge a > 1200 \}$

Q2) Find the names of all customers who have a loan of over \$1200

- $\{ \langle c \rangle \mid \exists l, b, a (\langle c, l \rangle \in \text{borrower} \wedge \langle l, b, a \rangle \in \text{loan} \wedge a > 1200) \}$

RELATIONAL ALGEBRA	RELATIONAL CALCULUS
It is a procedural method of solving the queries. We specify the sequence of operations to perform a particular request. It is prescriptive or rigid in nature I.e, it describes steps to perform a given task. The evaluation of the query depends upon the order of operations. It specifies operations performed on existing relations to obtain new relations. It is more closely associated with a programming language. The solution to the Database access problem Using a relational algebra is obtained by stating what is required and what are the steps to obtain that information. It is used as a vehicle for implementation of Relational Calculus. Relational algebra operators are used as a yardstick for measuring the expressive power of any given language. The queries are domain independent.	It is a non-procedural method of solving the queries. We specify the only what is required without bothering about the sequence of operations to perform that request. It is descriptive or straightforward in nature I.e. describe desired result. It does not depend on the order of operations. Operations are directly performed on the relations in form of formulas. It is more closely associated with a natural language. The solution to the database access problem using a relational calculus is obtained simply by stating what is required and letting the system find the answer. Relational Calculus queries are converted into equivalent required algebra format by stating Codd's Reduction Algorithm and then It is implemented with the help of relational algebra operators. A language is said to be complete if it is at least as powerful as the calculus that is, If any relation definable by some expression of the calculus is also definable by some expression the language in question. The queries are domain dependent.

Revision



Queries on Relational Algebra

Q) Consider the following schema:

Suppliers (sid, sname , address)

Parts (pid, pname, color)

Catalog (sid, pid, cost)

Queries on Relational Algebra- Projection

1. Write a query in relational algebra to display Sname of Suppliers?

$\pi_{\text{sname}}(\mathbf{Suppliers})$

2. Write a query in relational algebra to display pname and color of Parts?

$\pi_{\text{pname, color}}(\mathbf{Parts})$

3. Write a query in relational algebra to display cost of Catalog?

$\pi_{\text{cost}}(\mathbf{Catalog})$

Queries on Relational Algebra-Projection

4. Write a query in relational algebra to display sid and sname and address of Suppliers?

Ans:??????

5. Write a query in relational algebra to display sid and pid of Catalog?

Ans:??????

Queries on Relational Algebra- Selection

1. Write a query in relational algebra to display Pname of Parts whose color is Red?

$\Pi_{\text{pname}}(\sigma_{\text{color}='red'}(\text{parts}))$

2. Write a query in relational algebra to display Sname of Suppliers whose address is Thane?

$\Pi_{\text{sname}}(\sigma_{\text{address}='Thane'}(\text{Suppliers}))$

3. Write a query in relational algebra to display Pname of Parts whose color is Red and Blue?

$\Pi_{\text{pname}}(\sigma_{\text{color}='red' \text{ and } \text{color}='blue'}(\text{parts}))$

Queries on Relational Algebra- Selection

4. Write a query in relational algebra to display Sname of Suppliers whose sid is 50?

$\Pi_{\text{sname}}(\sigma_{\text{sid}=50}(\text{Suppliers}))$

5. Write a query in relational algebra to display Pname of Parts whose color is Green or White?

Ans:??

6. Write a query in relational algebra to display Cost of Catalog whose cost is 500 and 1000?

Ans:??

Queries on Relational Algebra- Joins

1. Write a query in relational algebra to display SID of suppliers whose name is 'Varun' and address is 'Chandigarh'?

$\pi_{sid}(\sigma_{sname='varun' \text{ and } address='chandigarh'}(\mathbf{Suppliers}))$

2. Write a query in relational algebra to display SID of suppliers who supplied some parts?

$\pi_{sid}(\mathbf{Catalog})$

Queries on Relational Algebra-Joins

3. Write a query in relational algebra to display Sname of suppliers who supplied some parts?

$\pi_{\text{sname}}(\sigma_{\text{s.sid}=\text{c.sid}}(\text{Supplier} \bowtie \text{Catalog}))$

or $\pi_{\text{sname}}(\text{Supplier} \bowtie \text{Catalog})$

4. Write a query in relational algebra to display Sname of suppliers who supplied some RED color parts?

$\pi_{\text{sname}}(\text{Supplier} \bowtie (\text{Catalog} \bowtie \sigma_{\text{color}='RED'}(\text{Parts})))$

Table Details

Member

Member ID	Name	Date of Birth
1	Alice	03/03/1995
2	Bob	11/07/1993
3	Charlie	21/10/1997
4	Mike	16/09/1992
5	Katie	21/10/1997

Book

Book ID	Name	Author
1	Inferno	Dan Brown
2	Ash	Malinda Lo
3	Fences	August Wilson
4	Origin	Dan Brown
5	Inheritance	Malinda Lo

Borrow

Member ID	Book ID	Borrow Date	Return Date
1	1	02/03/2020	12/03/2020
3	5	05/03/2020	15/03/2020
3	3	10/03/2020	20/03/2020
4	2	13/03/2020	23/03/2020
5	4	13/03/2020	13/03/2020

Queries on above Table

1. Write a query to display Member IDs of members who have borrowed books
2. Write a query to display Member IDs of members and the Book IDs of the books they have borrowed books.
3. Details of the members who were born on 21/10/1997.
4. Rename the Member relation as LibraryMember.
5. Perform cross product between Member and Borrow.
6. Perform natural join between Member and Borrow

Answers of Ques 1 and 2

$\pi_{Member\ ID}(Borrow)$

Member ID
1
3
4
5

$\pi_{Member\ ID,Book\ ID}(Borrow)$

Member ID	Book ID
1	1
3	5
3	3
4	2
5	4

Answers of Ques 3 and 4

$\sigma_{Date\ of\ Birth=21/10/1997}(Member)$

Member ID	Name	Date of Birth
3	Charlie	21/10/1997
5	Katie	21/10/1997

$\rho_{LibraryMember}(Member)$

LibraryMember

Member ID	Name	Date of Birth
1	Alice	03/03/1995
2	Bob	11/07/1993
3	Charlie	21/10/1997
4	Mike	16/09/1992
5	Katie	21/10/1997

Answers of Ques 5

Member × Borrow

Member ID	Name	Date of Birth	Member ID	Book ID	Borrow Date	Return Date
1	Alice	03/03/1995	1	1	02/03/2020	12/03/2020
1	Alice	03/03/1995	3	5	05/03/2020	15/03/2020
1	Alice	03/03/1995	3	3	10/03/2020	20/03/2020
1	Alice	03/03/1995	4	2	13/03/2020	23/03/2020
1	Alice	03/03/1995	5	4	13/03/2020	13/03/2020
2	Bob	11/07/1993	1	1	02/03/2020	12/03/2020
2	Bob	11/07/1993	3	5	05/03/2020	15/03/2020
2	Bob	11/07/1993	3	3	10/03/2020	20/03/2020
2	Bob	11/07/1993	4	2	13/03/2020	23/03/2020
2	Bob	11/07/1993	5	4	13/03/2020	13/03/2020
3	Charlie	21/10/1997	1	1	02/03/2020	12/03/2020
3	Charlie	21/10/1997	3	5	05/03/2020	15/03/2020
3	Charlie	21/10/1997	3	3	10/03/2020	20/03/2020
3	Charlie	21/10/1997	4	2	13/03/2020	23/03/2020
3	Charlie	21/10/1997	5	4	13/03/2020	13/03/2020
4	Mike	16/09/1992	1	1	02/03/2020	12/03/2020
4	Mike	16/09/1992	3	5	05/03/2020	15/03/2020
4	Mike	16/09/1992	3	3	10/03/2020	20/03/2020
4	Mike	16/09/1992	4	2	13/03/2020	23/03/2020
4	Mike	16/09/1992	5	4	13/03/2020	13/03/2020
5	Katie	21/10/1997	1	1	02/03/2020	12/03/2020
5	Katie	21/10/1997	3	5	05/03/2020	15/03/2020
5	Katie	21/10/1997	3	3	10/03/2020	20/03/2020
5	Katie	21/10/1997	4	2	13/03/2020	23/03/2020
5	Katie	21/10/1997	5	4	13/03/2020	13/03/2020

Ans of Ques 6

Member ⋈ Borrow

Member ID	Name	Date of Birth	Book ID	Borrow Date	Return Date
1	Alice	03/03/1995	1	02/03/2020	12/03/2020
3	Charlie	21/10/1997	5	05/03/2020	15/03/2020
3	Charlie	21/10/1997	3	10/03/2020	20/03/2020
4	Mike	16/09/1992	2	13/03/2020	23/03/2020
5	Katie	21/10/1997	4	13/03/2020	13/03/2020

Perform Union between Book IDs of the books borrowed by Charlie and Mike.

$$R1 = \pi_{Book\ ID}(\sigma_{Name="Charlie"}(Member \bowtie Borrow))$$

$$R2 = \pi_{Book\ ID}(\sigma_{Name="Mike"}(Member \bowtie Borrow))$$

$$R1 \cup R2$$

Book ID
2
3
5

Perform Intersection between Member IDs of the members who have borrowed both the books “Fences” and “Inheritance”

$$R1 = \pi_{Member\ ID}(\sigma_{Name="Fences"}(Book \bowtie Borrow))$$

$$R2 = \pi_{Member\ ID}(\sigma_{Name="Inheritance"}(Book \bowtie Borrow))$$

$$R1 \cap R2$$

Member ID
3

Perform Set difference between Member IDs of the members who have never borrowed books

$$R1 = \pi_{Member\ ID, Name, Date\ of\ Birth}(Member)$$

$$R2 = \pi_{Member\ ID, Name, Date\ of\ Birth}(Member \bowtie Borrow)$$

$$R1 - R2$$

Member ID	Name	Date of Birth
2	Bob	11/07/1993



Thank You